

Tempus Foundation Flow User Guide

Product Version 26.10
May 2026

©2011-2026 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used solely for personal, informational, and noncommercial purposes;
2. The publication may not be modified in any way;
3. Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
4. Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and/or replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

<u>About This Manual</u>	3
<u>Related Documents</u>	3
 <u>1</u>	
<u>Getting Started with Tempus Foundation Flows</u>	5
<u>Foundation Flow Overview</u>	6
<u>Setting up the Foundation Flow Environment</u>	6
<u>Using the Command Prompt</u>	6
<u>Using the GUI</u>	6
<u>Understanding the Foundation Flows Framework</u>	7
<u>Types of Foundation Flows</u>	8
<u>Launching the Foundation Flow</u>	9
<u>Using Code Generator</u>	10
<u>Advantages</u>	10
<u>Command-Line Options</u>	11
<u>Example Scripts</u>	12
<u>Using Predefined Scripts</u>	15
<u>Scripts for Steps in the Flow</u>	16
<u>Using Plug-in Variables for Settings and Reports</u>	16
 <u>2</u>	
<u>Tcl Variables</u>	19
<u>setup_mmmc.tcl File</u>	19
<u>setup_cpf.tcl File</u>	27
<u>setup_default.tcl File</u>	27

3

Timing Analysis Flow 49

Running Timing Analysis Using Foundation Flow 50

Optional Inputs 50

Flow Chart 50

Sample of Generated Code 51

Sample Script for Single View Non-MMMC flow 52

Running Concurrent and Distributed Multi-Mode Multi-Corner Analysis Using Foundation Flow 53

Running Distributed Static Timing Analysis Using Foundation Flow 54

4

ECO Fixing 55

Running ECO Fixing Using Foundation Flow 56

Before You Begin 56

Flow Chart 56

Sample of Generated Code 57

Sample Code of Generated run eco.tcl 58

Sample of Predefined Code 59

About This Manual

This manual describes the tasks in the recommended flow using the Cadence® Tempus™ Timing Solution software, also known as Tempus, through the timing and SI signoff.

The Tempus family encompasses the following products:

- Tempus L
- Tempus XL

For information about these products, see the "About Tempus Timing Solution Products and Product Options" section in the "Product and Licensing Information" chapter of the *Tempus User Guide*.

Related Documents

For more information about Tempus, see the following documents. You can access these and other Cadence documents with the Cadence Help online documentation system.

- **Tempus Text Command Reference**
Describes the Tempus text commands, including syntax and examples.
- **Tempus Menu Reference**
Describes Tempus graphical user interface.
- **Tempus User Guide**
Provides information about using various Tempus features.
- **Tempus Known Problems and Solutions**
Describes important Cadence Change Requests (CCRs) for Tempus, including solutions for working around known problems.

Tempus Foundation Flow User Guide

About This Manual

Getting Started with Tempus Foundation Flows

- [Foundation Flow Overview](#)
- [Setting up the Foundation Flow Environment](#)
 - [Using the Command Prompt](#)
 - [Using the GUI](#)
- [Understanding the Foundation Flows Framework](#)
- [Types of Foundation Flows](#)
- [Launching the Foundation Flow](#)
 - [Using Code Generator](#)
 - [Advantages](#)
 - [Command-Line Options](#)
 - [Example Scripts](#)
 - [Using Predefined Scripts](#)
 - [Scripts for Steps in the Flow](#)
 - [Using Plug-in Variables for Settings and Reports](#)

Foundation Flow Overview

The Foundation Flows are the Cadence-recommended procedures for performing MMMC Signoff, CPF-based MMMC Signoff, and Single view non-MMMC flow using the Tempus software. Cadence provides a set of scripts that give you an easy way to automate these flows.

The Foundation Flows are a starting point for building Signoff Analysis environment, but you can augment them with design-specific content.

Setting up the Foundation Flow Environment

You can create a foundation flow environment in two ways:

Using the Command Prompt

You can extract the foundation flows by running the `write_flow_template` command.

1. Create a run directory and change directory to it

```
mkdir your_work_area/RUN1  
cd your_work_area/RUN1
```

2. Run the `write_flow_template` command. This command copies the Tempus Foundation Flows templates into the directory you specify (or to the current directory if you do not specify a directory with the `-directory` parameter):

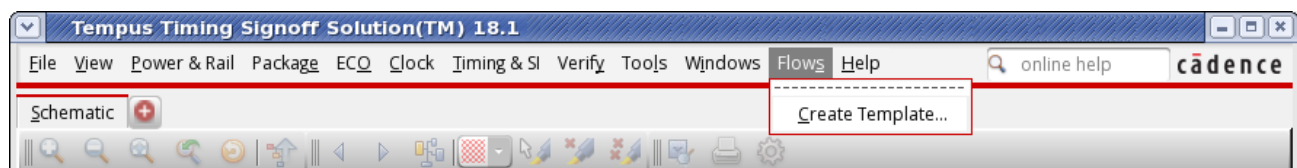
```
write_flow_template -directory
```

Using the GUI

You can select an option to save the templates in a new directory.

1. Select *Flows* > Create Template.

Figure 1-1 The Flows Menu

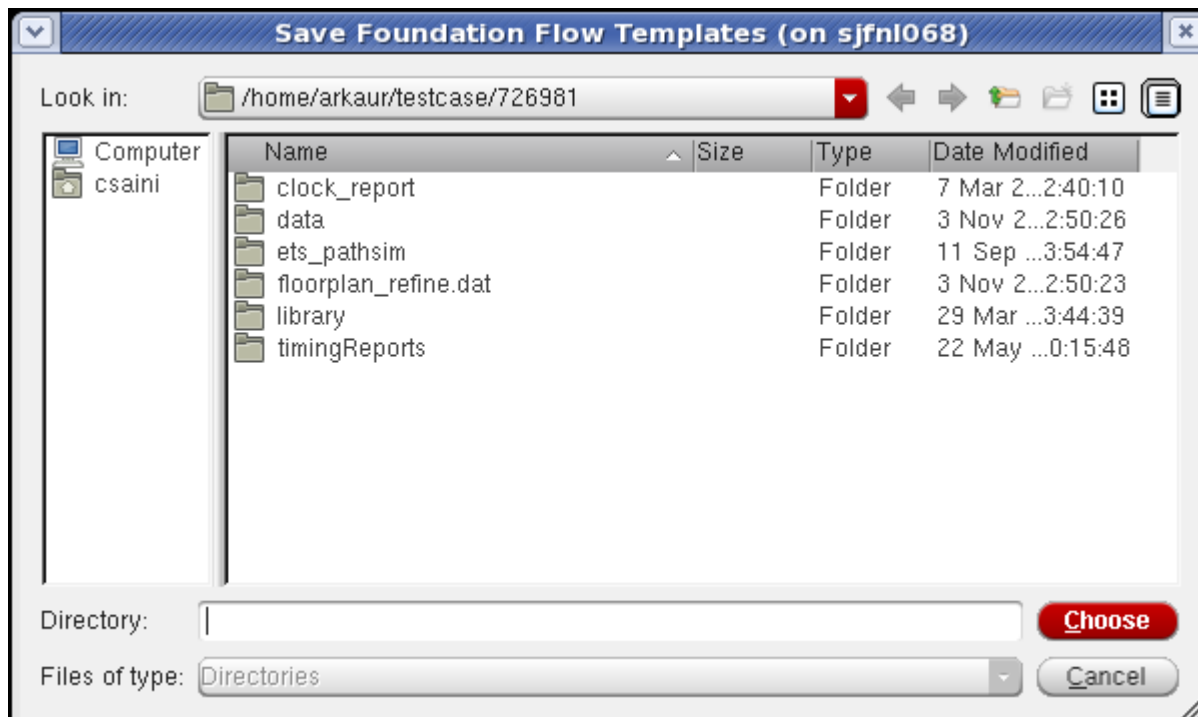


Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

The Save Foundation Flow Templates dialog box prompts you to choose the directory to save the templates. A message confirms that the foundation flow templates have been saved at the selected location.

Figure 1-2 The Save Foundation Flow Templates Form



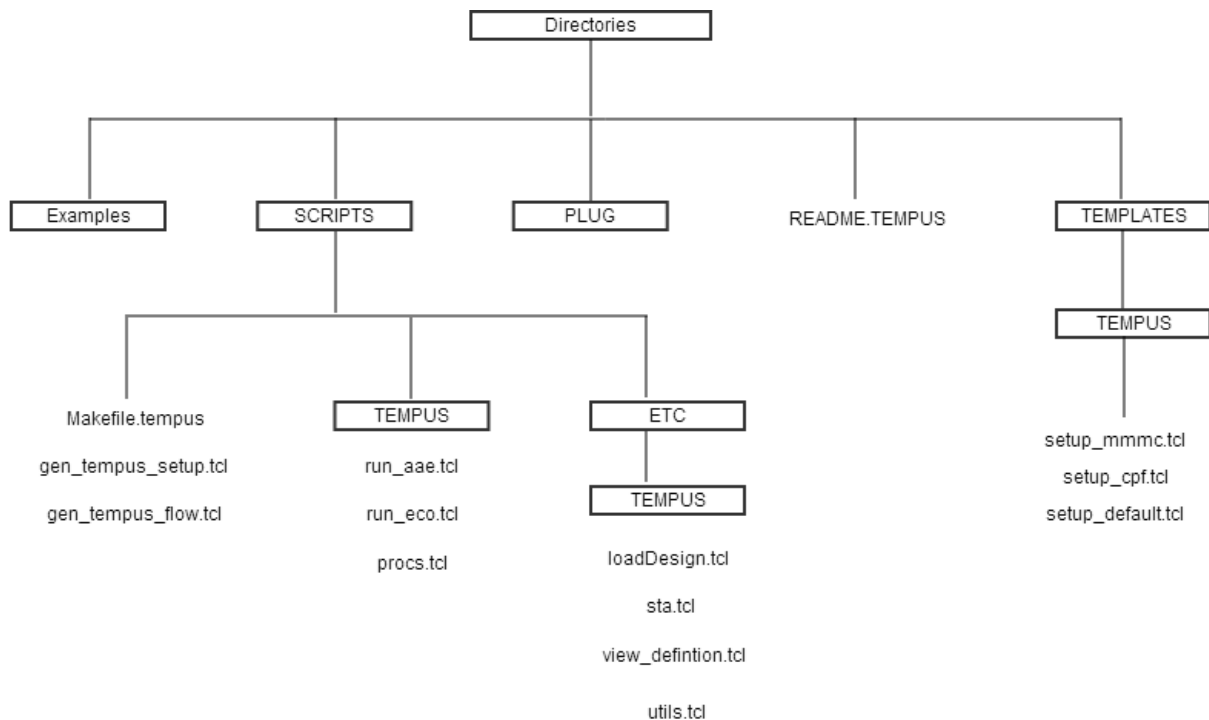
Understanding the Foundation Flows Framework

The foundation flow framework is described below:

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

Figure 1-3 The Foundation Flow Framework



The Tempus foundation flow directory consists of:

- **EXAMPLES** - a folder containing example scripts for a sample design. These scripts have design specific content that explains how to set up the scripts.
- **SCRIPTS** - a folder containing the Tempus foundation flow related scripts. The Tempus directory in the SCRIPTS directory contains all the `run_*.tcl` files.
- **PLUG** - Contains Tempus plug-in scripts. These scripts provide an easy method to expand and customize the basic flows.
- **README.Tempus** - a file containing the directory structure of the foundation flow, the content of each directory, and how to use the foundation flows.
- **TEMPLATES** - a folder containing the `setup_mmmc.tcl`, `setup_cpf.tcl`, and `setup_default.tcl` files that defines the variables to use in the MMMC analysis, low power, and single view non MMMC flow.

Types of Foundation Flows

In Tempus, you have three stand-alone foundation flows. These are:

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

- MMMC Signoff flow
- CPF-based MMMC Signoff flow
- Single-view non MMMC flow

Each flow can either load an Innovus database or can source some other input files. For non-Innovus based flow, various input files such as design data, library information are sourced and the design is loaded. You can also load some other input files such as SDF, TWF, or IR drop files for all type of flows.

Each flow has its own setup file to configure the design. You can use one of the following pre-defined scripts to run these flows:

- timing analysis: `run_aae.tcl`
- tempus signoff optimization fixing: `run_eco.tcl`

For example, the Single view non MMMC flow has `setup_default.tcl` to configure its design, and `run_aae.tcl` to run the flow.

Launching the Foundation Flow

You can launch the Foundation flow from `WORK/dir`:

```
% mkdir WORK
```

The Foundation flow requires a `setup.tcl` as input. To create this file, copy `TEMPLATES/TEMPUS/setup_*.tcl` to `WORK/setup.tcl` and edit the file:

1. For MMMC flow, copy `TEMPLATES/TEMPUS/setup_mmmc.tcl`
2. For CPF flow, copy `TEMPLATES/TEMPUS/setup_cpf.tcl`
3. For single view non-MMMC flow, copy `TEMPLATES/TEMPUS/setup_default.tcl`

Note: If `setup.tcl` is not copied in `WORK/dir`, use "`setenv FF_SETUP_PATH <path>`" to specify the directory, which contains the `setup.tcl` file before launching `TEMPUS`.

There are two ways to run the Foundation flow in Tempus:

- Using Code Generator
- Using Predefined Scripts

Using Code Generator

The foundation flow scripts are difficult to read and debug. This hinders those users who would like to use the scripts as a reference flow as well as for auditing purposes where readability is important. Tempus provides a code generator to enable users to quickly generate scripts to load their design data, setup their timing environment, and execute the recommended timing and SI flow.

The foundation flow is in the SCRIPTS directory and the code generator is `SCRIPTS/gen_tempus_flow.tcl`.

Advantages

The following are the advantages of the Code Generator:

- Through the Code Generator, the scripts will be "generated" instead of being "executed".
- All flow will have the command sequence based on the user settings. A clean, flat script will be generated for conditional code that is based on static user settings (for example, `setup.tcl`).
- All conditional code branches that require run time specific data will be generated as conditionals (that is, they cannot be "flattened").

For example, since analysis view can be altered via user plug-ins, checks must be made to determine which corners are active:

```
set vars(active_rc_corners) [list]
foreach view [concat [all_setup_analysis_views]
[all_hold_analysis_views]] {
    set corner [get_delay_corner [get_analysis_view $view -delay_corner] -
rc_corner]
    if {[lsearch $vars(active_rc_corners) $corner] == -1 } {
        lappend vars(active_rc_corners) $corner
    }
}
```

The following procedures will be covered in `procs.tcl`.

Procedure	Type	Description
system_info		Reports system info of the host machine (<code>vars(report_system_info)</code>)
source_file	file	Wrapper around 'source' to catch syntax errors
source_plug	plug_in	Loads one or more plug-in scripts, catch errors, and so on.

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

report_time		Reported elapsed CPU time (vars(report_run_time))
-------------	--	---

TCL Command

Usage: tclsh ../SCRIPTS/gen_tempus_flow.tcl [-d output_dir]

Command-Line Options

The following are the command line options:

Command	Description
-l	Prints each Tempus command out on a single line. By default, the Tempus commands may be printed across lines with an approximate line size of 80-90 characters.
-h -help	Print this message
-f -flat	Flatten/unroll levels
-d -dir	The code generator creates a design specific set of run scripts in a FF directory in the local user directory. You can optionally change the name of the output directory using this option.
-p -pseudo	Provides pseudo run script of Tempus instead of whole design specific FF environment. This option can be used as follows: ■ -p: An existing setup.tcl file is defined. File checking step is skipped to generate the pseudo code. ■ -p default: Generates a reference setup.tcl and pseudo code for the default flow. ■ -p mmmc: Generates a reference setup.tcl and pseudo code for the MMMC flow. ■ -p cpf: Generates a reference setup.tcl and pseudo code for the CPF MMMC flow. To improve usability, the Tempus foundation flow allows the use of pseudo mode. In this mode, the foundation flow does not require any input data; the software directly dumps the Tempus run script. You can edit the script manually after checking the Tempus flow, and then replace the pseudo file name with the real data file name before running Tempus.

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

<code>-s -setup</code>	Provide the directory containing the <code>setup.tcl</code> setup file required to run the Foundation Flow. Default is ".".
--------------------------	--

Usage

Below are the steps to use the Code Generator:

1. Dump the foundation flow code generator from the Tempus interface or download from the download manager.
 - ❑ GUI: Tools > Foundation Flow Template > Save
 - ❑ TUI: `write_flow_template -directory directory`
2. Create a `setup.tcl` using either the foundation flow wizard or using `gen_tempus_setup.tcl`.
 - ❑ To create using `gen_tempus_setup.tcl`,
 - load an existing database
 - and "`source <directory>/SCRIPTS/gen_tempus_setup.tcl`" (the existing data base should be the starting point for the foundation flow)
 - Rename the files to `setup.tcl`.
3. Run the code generator: `tclsh directory /SCRIPTS/gen_tempus_flow.tcl -doutput_dir_name`
4. Execute the flow: `make`

Note: If you are using Innovus, source `../SCRIPTS/gen_tempus_setup.tcl` to generate the `setup.tcl` file.

Example Scripts

Following are the examples of the code generation:

STA and SI Flow

```
% tclsh SCRIPTS/gen_tempus_flow.tcl -s ../
<-----
<FF> TCL VERSION -> 8.5
-----
<FF> SETUP PATH == ../
```

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

```
<FF> LOADING CONFIGURATION FILES:
<FF> setup_tcl -> ../setup.tcl
<FF> Generating script for mode: single
<FF> RUNNING SETUP CHECK
<FF> WARNING: Operating Condition not defined flow will continue
```

```
-----
<FF> Generating single Makefile ...
-----
-----
-----
```

```
<FF> CODE GENERATION COMPLETE
-----
```

MMMC Signoff Flow

```
% tclsh SCRIPTS/gen_tempus_flow.tcl -s ../
```

```
-----
<FF> TCL VERSION -> 8.5
-----
```

```
<FF> SETUP PATH == ../
<FF> LOADING CONFIGURATION FILES:
<FF> setup_tcl -> ../setup.tcl
<FF> Generating script for mode: mmmc
<FF> RUNNING SETUP CHECK
<FF> Loading Innovus database so file checking suppressed
<FF> Processing view io+typ-rcMax ...
<FF> Processing view core+typ-rcMax ...
<FF> Processing view io+typ-rcMin ...
<FF> Processing view core+best-rcMax ...
```

```
-----
<FF> Generating mmmc Makefile ...
-----
-----
-----
```

```
<FF> CODE GENERATION COMPLETE
-----
```

CPF-Based MMC SignOff Flow

```
% tclsh SCRIPTS/gen_tempus_flow.tcl -s ../
```

```
-----
<FF> TCL VERSION -> 8.5
-----
```

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

```
<FF> SETUP PATH == ..
<FF> LOADING CONFIGURATION FILES:
<FF> setup_tcl -> ../setup.tcl
<FF> Generating script for mode: mmmc
<FF> RUNNING SETUP CHECK
<FF> Processing view A1_WC ...
<FF> Processing view A1_WC_scan ...
<FF> Processing view A1_BC ...
<FF> Processing view A1_BC_scan ...

-----
<FF> Generating mmmc Makefile ...

-----
<FF> CODE GENERATION COMPLETE
-----
```

Results

The code generator default to creating an "instance" of the foundation flow in a "FF" directory in the local user directory. You can optionally override this using the option `-d directory`.

Here is a list of the directories created by the code generator:

- `db`s: Contains the design saved during foundation flow run.
- `log`: Contains the log files
- `rpt`: Contains all the timing reports
- `FF`: Tcl file generated by the code generator script. It contains separate files for globals and variables.

The following files are generated as a result of running the setup generator:

- For MMC design, `setup.mmc.tcl` is generated.
- For CPF design, `setup.cpf.tcl` is generated.
- For single corner non-MMC design, `setup.auto.tcl` is generated.

Pseudo Mode Generation Through ‘-p’ Option

When “default” is given FF will generate a ‘STA and SI Flow’ pseudo scripts without the need of a predefined `setup.tcl`:

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

```
Default Flow ==
>$ tclsh ../SCRIPTS/gen_tempus_flow.tcl -p default
-----
<FF> TCL VERSION -> 8.4
-----
<FF> LOADING CONFIGURATION FILES:
<FF> setup_tcl -> ./setup.tcl
<FF> Generating script for mode: aae
-----
<FF> Generating aae Makefile ...
-----
-----
<FF> CODE GENERATION COMPLETE
-----
```

Using Predefined Scripts

```
% cd WORK
% tempus -nowin -64 -init ../SCRIPTS/TEMPUS/run_aae.tcl # for timing analysis
% tempus -nowin -64 -init ../SCRIPTS/TEMPUS/run_eco.tcl # for tempus signoff
optimization fixing as necessary
```

You can also use the SCRIPTS/Makefile.tempus to call:

```
% cd WORK
% cp ../SCRIPTS/Makefile.tempus Makefile ( modify as needed )
% make
```

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

Scripts for Steps in the Flow

Flow	Script	Description
MMMC Signoff flow	setup_mmmc.tcl run_aae.tcl loadDesign.tcl sta.tcl run_eco.tcl utils.tcl	Configures the design Runs the MMMC flow Runs the ECO fixing Foundation Flow utilities
CPF MMMC Signoff flow	setup_cpf.tcl run_aae.tcl loadDesign.tcl sta.tcl run_eco.tcl utils.tcl	Configures the design Runs the CPF-based MMMC flow Runs the ECO fixing Foundation Flow utilities
Single View non MMMC flow	setup_default.tcl run_aae.tcl run_eco.tcl utils.tcl	Configures the design template Runs the flow Foundation Flow utilities

Note: Before launching Tempus, either copy `setup.tcl` into the `WORK/` directory or use `setenv FF_SETUP_PATH <path>` to specify the directory, which contains the `setup.tcl` file.

Using Plug-in Variables for Settings and Reports

To allow customized timing analysis flow, plug-in variables are included to allow you to apply your own settings and generate specific reports. The entire timing analysis flow is divided into the following steps:

- Design import
- Parasitic load
- STA
- Report Generation

For each step in the flow, there are plug-ins before and after the main command. For example, in the reporting step, there is a plug-in before and after the sample timing reports generating command. The plug-ins are called `tempus_pre_ <step>` and `tempus_post_ <step>`.

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

So, in the case of the report generation step, the plug-in will be called `vars(tempus_pre_rpt_gen)` and `vars(tempus_post_rpt_gen)`.

These variables point to a user-generated TCL file containing Tempus system commands that are run in addition to the basic flow commands.

You can also have your own flow steps to replace the standard flow steps by setting the plug-in `tempus_replace_ <step>`, for example,

```
set vars(tempus_pre_rpt_gen) "./my_rpt_gen.rpt".
```

The following plug-ins are supported.

Plug-in Variable Name	Value Type	Usage Description
<code>tempus_pre_design_import</code>	string	<path to plug-in tcl file>,executed before design import
<code>tempus_replace_design_import</code>	string	<path to plug-in tcl file>,executed to replace design import
<code>tempus_post_design_import</code>	string	<path to plug-in tcl file>,executed after design import
<code>tempus_pre_parasitic_load</code>	string	<path to plug-in tcl file>,executed before parasitic load
<code>tempus_replace_parasitic_load</code>	string	<path to plug-in tcl file>,executed to replace parasitic load
<code>tempus_post_parasitic_load</code>	string	<path to plug-in tcl file>,executed after parasitic load
<code>tempus_pre_sta</code>	string	<path to plug-in tcl file>,executed before first timing report
<code>tempus_replace_sta</code>	string	<path to plug-in tcl file>,executed to replace first timing report
<code>tempus_post_sta</code>	string	<path to plug-in tcl file>,executed after first timing report
<code>tempus_pre_rpt_gen</code>	string	<path to plug-in tcl file>,executed before reports generation
<code>tempus_post_rpt_gen</code>	string	<path to plug-in tcl file>,executed to replace reports generation

Tempus Foundation Flow User Guide

Getting Started with Tempus Foundation Flows

tempus_post_rpt_gen	string	<path to plug-in tcl file>,executed before reports generation
---------------------	--------	---

You can also skip one step by setting plug-in `tempus_skip_ <step>`, for example, if a design does not have a SPEF file to load in, then you can add the following statement in the `setup.tcl` file:

```
set vars(tempus_skip_parasitic_load) true
```

Plug-in Variable Name	Value Type	Usage Description
tempus_skip_design_import	boolean	set to true to skip design import
tempus_skip_parasitic_load	boolean	set to true to skip parasitic load
tempus_skip_sta	boolean	set to true to skip first timing report
tempus_skip_rpt_gen	boolean	set to true to skip reports generation

Note that this flow does not include error checking. However, if any error occurs, it will be reported, and the software will exit and save the databases, as required.

Tcl Variables

The Foundation Flow requires a configuration file (`setup.tcl`) that defines the necessary information to execute the flow including pointers to design data and command option information. Under the `TEMPLATES` directory, there are three sample setup files for different flows. You can copy these setup files to the running directory, rename these setup files to `setup.tcl`, and modify these as required.

- **setup_mmmc.tcl File**

Defines the variables to use in the MMMC analysis. This script is unique for each design and is the only script that you must edit. It is the source of all input flows, and is a superset of the configuration file that includes design data, library information, and flow control.

- **setup_cpf.tcl File**

This template is for the low power design specially, which uses the `cpf` file as the power domain definition file.

- **setup_default.tcl File**

This template is used for the single view non MMMC flow; if you have only one view to analyze and do not want to define view definition.

Note: Two sample `setup.tcl` files have been provided that are able to run through. The `setup.tcl` files - one for single view non MMMC flow is located at `EXAMPLES/TEMPUS/dtmf/WORK` directory and the other for MMMC flow is available at `EXAMPLES/TEMPUS/dtmf_mmmc/WORK` directory. The required libraries, netlists, and SPEF data is also included in the `EXAMPLES` directory.

setup_mmmc.tcl File

Defining Variables and Specifying Values for Timing Environment

Variables defining the scripts directory, output reports, saved session directories and environment settings.

Variable Name	Value Type	Usage Description

Tempus Foundation Flow User Guide

Tcl Variables

Variables to Specify file and saved session directory setting		
flow	string	supported flow -> single, mmmc and cpf
script_root	string	foundation flow scripts dir (default is "../SCRIPTS")
dbs_dir	string	Saved session directory (default is dbs)
Variables for Design Initialization		
si_analysis	string	set "true" to include si analysis (default is "true")
aocv_flow	string	set "true" to switch on the aocv analysis
pba_analysis	string	set "true" to switch on path based analysis for violated paths
aocv_pba_analysis	string	set "true" to switch on aocv path based analysis for violated paths
physical_data	string	set "true" to include physical data (default setting is "false" to just load the timing related data)
Variables to Specify All the Setup and Hold Analysis Views		
setup_analysis_views	string	List all analysis views in this design, e.g. "func_slow test_slow"
hold_analysis_views	string	List all analysis views in this design, e.g. "func_fast test_fast"
Variable to Specify Concurrent and Distributed MMMC		
combine_views	string	User can also give N views here to enable concurrent + distribute MMMC flow N is the maximum of views to on the same remote
Variables to Specify the SPEF Files for Each RC Corner		

Tempus Foundation Flow User Guide

Tcl Variables

<code>\$rc_corner,spef_file</code>	string	Load all the used .spef for the rc_corner, e.g. <code>vars(slow_rc,spef)</code> . The default spef file name will be <code>\$rc_corner.spef.gz</code>
------------------------------------	--------	---

Variables Defining All the Library Sets, RC Corners, Delay Corners, Constraint Modes and Virtual Operating Conditions

<code>library_sets</code>	string	List all the library sets used in this design, e.g. "slow fast"
<code>rc_corners</code>	string	List all the rc corners used in this design, e.g. "slow_rc fast_rc"
<code>delay_corners</code>	string	List all the delay_corner used in this design, e.g. "slow_dc fast_dc"
<code>constraint_modes</code>	string	List all the constraint modes used in this design, e.g. "func test"
<code>vir_op_conds</code>	string	List all the virtual condition need to be created in this design, e.g. "BCCOM_slow WCCOM_slow"

Variables to Specify the Library Files for Each Library Set

<code>\$library_set,timing</code>	string	Load all the used .libs for the library_set, e.g. <code>vars(slow,timing)</code>
<code>\$library_set,si</code>	string	Load all the used .cdfs for the library_set, e.g. <code>vars(slow,si)</code>
<code>\$library_set,aocv</code>	string	Load all the used aocv libs for the library_set, e.g. <code>vars(slow,aocv)</code>

Variables to Specify the Binding Library File, Process Value, Voltage and Temperature for the Created Virtual Operating Conditions

<code>\$vir_op_cond,lib_file</code>	string	give the binding library file name, e.g. <code>vars(BCCOM_slow,lib_file)</code>
-------------------------------------	--------	--

Tempus Foundation Flow User Guide

Tcl Variables

<code>\$vir_op_cond,P</code>	string	give the binding process value, e.g. <code>vars(BCCOM_slow,P)</code>
<code>\$vir_op_cond,V</code>	string	give the binding voltage value, e.g <code>vars(BCCOM_slow,V)</code>
<code>\$vir_op_cond,T</code>	string	give the binding temperature value, e.g. <code>vars(BCCOM_slow,T)</code>
Variables to Specify the Binding rc_corner, Library Set, operating condition and opcond library name "for Each delay_corner"		
<code>\$delay_corner,library_set</code>	string	give the binding library_set name, e.g. <code>vars(slew_dc,library_set)</code>
<code>\\$delay_corner,library_set,late</code>	string	give the binding library_set name, e.g. <code>vars(slew_dc,library_set,late)</code>
<code>\\$delay_corner,library_set,early</code>	string	give the binding library_set name, e.g. <code>vars(slew_dc,library_set,early)</code>
<code>\$delay_corner,rc_corner</code>	string	give the binding rc_corner name, e.g. <code>vars(slew_dc,rc_corner)</code>
<code>\$delay_corner,opcond</code>	string	give the binding opcond name (optional)
<code>\$delay_corner,opcond_lib</code>	string	give the binding opcond library name (optional)
Variables to Specify the Binding SDC files for Each constraint_mode		
<code>\$constraint_mode,signoff_sdc</code>	string	give the binding sdc_file name, e.g <code>vars(func,sdc_file)</code>
Variables to Specify the Binding Constraint Mode and Delay Corner for Each View		
<code>\$analysis_view,constraint_mode</code>	string	give the binding constraint mode name, e.g <code>vars(func_slow,constraint_mode)</code>

Tempus Foundation Flow User Guide

Tcl Variables

<code>\$analysis_view, delay_corner</code>	string	give the binding delay corner name, e.g. <code>vars(func_slow, delay_corner)</code>
Variable to Specify a Predefined View Definition File		
<code>view_tcl</code>	string	if a predefined view definition file is provided, the above MMMC settings are not required
Variable to Define the Type of Distributed Processing		
<code>distribute</code>	string	the value can be <local custom lsf rsh>
<code>remote_hosts</code>	string	set to value = Total(setup views + hold views) machines for distribute all analysis views onto different machines; it can also be set to any value < Total(setup views + hold views) the runtime is longer (will be ready before 91rtm, SI results in this mode is not accurate now)
Variables for RSH Mode of Distributed Processing		
<code>rsh, host_list</code>	string	provide the host name
<code>distribute_timeout</code>	string	provide the time limit for a rsh remote host to respond, the tool's default is 5 sec
Variables for Custom Mode of Distributed Processing		
<code>custom, script</code>	string	provide the custom script
Variables for LSF Mode of Distributed Processing		
<code>lsf, queue</code>	string	provide the lsf queue name
<code>lsf, resource</code>	string	provide the lsf resource name
<code>lsf, args</code>	string	provide other lsf args
Variable for Specifying Distributed Static Timing Analysis (DSTA) Capability		

Tempus Foundation Flow User Guide

Tcl Variables

dsta	string	# by default DMMMC flow will be generated # set to true to switch on DSTA flow
Variable for Specifying Multi-Threading Settings		
local_cpus	string	specify the number of local CPUs used for multithreading runs if DMMMC is enabled, the value means number of CPUs per remote host
Variables for Specifying SDF and IR-Drop Files (Optional)		
\$analysis_view,sdf_file,full	string	provide the full sdf file name (optional)
\$analysis_view,sdf_file,incr	string	provided the sdf file name of incr delay (optional)
\$delay_corner,ir_drop_early	string	provide the ir-drop files name for early path (optional)
\$delay_corner,ir_drop_late	string	provide the ir-drop files name for late path (optional)
Variables to Specify Timing Derate and Uncertainty Values		
\$delay_corner,cell_data_late	string	set derate value for cell for data late path (optional)
\$delay_corner,cell_data_early	string	set derate value for cell for data early path (optional)
\$delay_corner,cell_clock_late	string	set derate value for cell for clock late path (optional)
\$delay_corner,cell_clock_early	string	set derate value for cell for clock early path (optional)
\$delay_corner,net_data_late	string	set derate value for net for data late path (optional)
\$delay_corner,net_data_early	string	set derate value for net for data early path (optional)
\$delay_corner,net_clock_late	string	set derate value for net for clock late path (optional)

Tempus Foundation Flow User Guide

Tcl Variables

<code>\$delay_corner,net_clock_early</code>	string	set derate value for net for clock early path (optional)
<code>\$delay_corner,derate_tcl</code>	string	alternatively, users can specify a derating files (optional)
<code>\$delay_corner,clock_uncertainty,setup</code>	string	set clock uncertainty for setup check (optional)
<code>\$delay_corner,clock_uncertainty,hold</code>	string	set clock uncertainty for setup check (optional)
Variables for MMMC ECO Fix		
<code>eco_buffers</code>	string	provide buffers for eco fix if <code>\$vars(eco_fix)</code> set to true
<code>eco_targets</code>	string	provide eco optimization targets. Choices include hold, drv, setup and leakage default is hold; Tempus Signoff Optimization allows multiple incremental ECOs, e.g. "drv hold setup". Leakage opt needs separate session.
Variables for Plug-In Tcl Scripts		
<code>tempus_pre_design_import</code>	string	<path to plug-in tcl file>,executed before design import (optional)
<code>tempus_replace_design_import</code>	string	<path to plug-in tcl file>,executed to replace default design import (optional)
<code>tempus_post_design_import</code>	string	<path to plug-in tcl file>,executed after design import (optional)
<code>tempus_pre_parasitic_load</code>	string	<path to plug-in tcl file>,executed before parasitic load (optional)
<code>tempus_replace_parasitic_load</code>	string	<path to plug-in tcl file>,executed to replace parasitic load (optional)

Tempus Foundation Flow User Guide

Tcl Variables

tempus_post_parasitic_load	string	<path to plug-in tcl file>,executed after parasitic load (optional)
tempus_pre_sta	string	<path to plug-in tcl file>,executed before first report timing (optional)
tempus_replace_sta	string	<path to plug-in tcl file>,executed to replace first report timing (optional)
tempus_post_sta	string	<path to plug-in tcl file>,executed after first report timing (optional)
tempus_pre_si	string	<path to plug-in tcl file>,executed before SI analysis (optional)
tempus_replace_si	string	<path to plug-in tcl file>,executed to replace SI analysis (optional)
tempus_post_si	string	<path to plug-in tcl file>,executed after SI analysis (optional)
tempus_pre_rpt_gen	string	<path to plug-in tcl file>,executed before reports generation (optional)
tempus_replace_rpt_gen	string	<path to plug-in tcl file>,executed to replace reports generation (optional)
tempus_post_rpt_gen	string	<path to plug-in tcl file>,executed after reports generation (optional)
tempus_pre_eco	string	<path to plug-in tcl file>,executed before start eco opt (optional)
tempus_replace_eco	string	<path to plug-in tcl file>,executed to replace eco opt (optional)
tempus_post_eco	string	<path to plug-in tcl file>,executed after eco opt (optional)

Tempus Foundation Flow User Guide

Tcl Variables

Variables to Skip Each Default Steps		
tempus_skip_design_import	string	set to "true" to skip design import (optional)
tempus_skip_parasitic_load	string	set to "true" to skip parasitic load (optional)
tempus_skip_sta	string	set to "true" to skip first report timing (optional)
tempus_skip_si	string	set to "true" to skip SI analysis (optional)
tempus_skip_rpt_gen	string	set to "true" to skip report generation (optional)

setup_cpf.tcl File

When compared to the general MMMC design, low power design has CPF as the input for the power domain and analysis view definition. Besides the CPF file, you need to bind `rc_corner` to the corresponding delay corner to complete the view definition:

Variable Name	Value Type	Usage Description
cpf_file	string	Provides the full cpf file name
rc_corners	string	Lists all the rc corners used in this design, for example, "slow_rc fast_rc". If you want to use the default rc corner, set the variable to "default_rc_corner"
\${analysis_view}_dc,rc_corner	string	Specifies the binding rc_corner name, for example, vars(func_slow_dc,rc_corner)

setup_default.tcl File

Use the `setup_default.tcl` template if you have a single view for analysis, and do not want to create view definitions.

Defining Variables and Specifying Values for Timing Environment

Tempus Foundation Flow User Guide

Tcl Variables

Variables defining the scripts directory, output reports, saved session directories and environment settings.

Variable Name	Value Type	Usage Description
Design Initialization Variables		
si_analysis	string	set "true" to include si analysis (default is "true")
aocv_flow	string	set "true" to switch on the aocv analysis
pba_analysis	string	set "true" to switch on path based analysis for violated paths
aocv_pba_analysis	string	set "true" to switch on aocv path based analysis for violated paths
physical_data	string	set "true" to include physical data (default setting is "false" to just load the timing related data)
Design Import Variables		
load_edt_db	string	set "true" for loading design from EDI DB design saved from TEMPUS session can be restored in the same way
edi_db_name	string	EDI database or a conf file (required if load_edt_db is true, default is "DBS/signoff.enc")
edi_oa_name	string	EDI OA database (required if load_edt_db is true, and the database is in OA format
spef_file	string	SPEF files
Variables to Load Library and Design Data		
Load in single library set		

Tempus Foundation Flow User Guide

Tcl Variables

libs,timing	string	List of slow (or fast) .libs
Load in AOCV library		
libs,aocv	string	List of aocv libraries
netlist	string	Input netlist
lef_files	string	Input lef file (optional, required only if need to load physical data)
def_files	string	Input def file (optional, required only if need to load physical data)
oa_Ref	string	Input OA_reference_libs (optional, required only if need to load physical OA data)
design	string	top design name
signoff_sdc	string	signoff SDC file
Variable to Specify Multi-Threading Settings		
local_cpus	string	specify the number of local CPUs used for multithreading runs
Variables to Load Additional Files		
sdf_file,full	string	provide the full sdf file name (optional)
sdf_file,incr	string	provided the sdf file name of incr delay (optional)
ir_drop	string	provide the ir-drop files name (optional)
Variables for ILM Support		
read_ilm,cell_list	string	provide the corresponding ilm cell name list if ILM herarchical flow is used (optional)
read_ilm,dir_list	string	provide the ilm dir list for the above dir (optional)

Tempus Foundation Flow User Guide

Tcl Variables

write_ilm_dir	string	provide the written ilm dir name if want to dump ILM data (optional)
Variables for Timing Analysis Settings		
op_cond,name	string	provide the condition name will be used for the design,e.g "WCCOM" (optional)
op_cond,lib	string	provide the library name where the condition is defined (optional)
op_cond_max,name	string	provide the max condition name will be used for the design,e.g "WCCOM" (optional)
op_cond_max,lib	string	provide the max library name where the condition is defined (optional)
op_cond_min,name	string	provide the min condition name will be used for the design,e.g "WCCOM" (optional)
op_cond_min,lib	string	provide the min library name where the condition is defined (optional)
Variables to Specify STA Settings		
analysis_mode	string	value can be chosen from single,bcwc and onChipVariation default value for signoff is "onChipVariation"
cppr	string	value can be none,setup,hold and both default value for signoff is "both"
clock_uncertainty,setup	string	set clock uncertainty for setup check (optional)
clock_uncertainty,hold	string	set clock uncertainty for hold check (optional)

Tempus Foundation Flow User Guide

Tcl Variables

Variables to Specify Timing Derate Values		
<code>derate,cell_data_late</code>	string	set derate value for cell for data late path (optional)
<code>derate,cell_data_early</code>	string	set derate value for cell for data early path (optional)
<code>derate,cell_clock_late</code>	string	set derate value for cell for clock late path (optional)
<code>derate,cell_clock_early</code>	string	set derate value for cell for clock early path (optional)
<code>derate,net_data_late</code>	string	set derate value for net for data late path (optional)
<code>derate,net_data_early</code>	string	set derate value for net for data early path (optional)
<code>derate,net_clock_late</code>	string	set derate value for net for clock late path (optional)
<code>derate,net_clock_early</code>	string	set derate value for net for clock early path (optional)
<code>derate_tcl</code>	string	alternatively, you can specify a derating files (optional)
Variables to Specify SI Settings		
<code>si_rpt_prefix</code>	string	set the SI report prefix, default is "tempus"
<code>process</code>	string	set SI analysis process,e.g. "65nm"
<code>noise,delta_absolute</code>	string	set the noise delay threshold,the tool's default value is 0.0

Tempus Foundation Flow User Guide

Tcl Variables

twf,conv_iter	string	set the timing window convergence,the tool's default setting is 2
twf_file	string	set the load in twf file name (optional),otherwise infinite twf is used to start
Variables to Specify Advanced SI Settings		
noise,delay_peak	string	noise peak for noise-on-delay filtering,tool's default is 20% of the VDD
noise,glitch_peak	string	noise peak for glitch filtering,tool's default is 20% of the VDD
vir_attacker,mode	string	specify whether to use the current-matching based method for virtual attackers,default setting is "current"
vir_attacker,factor	string	multiplication factor of virtual attacker,tool's default is 1
vir_attacker,gtol	string	virtual attacker grounding threshold, default is 2% of VDD
vir_attacker,switch_prob	string	virtual switch probability,only valid with current mode VA; tool's default is 0.3333
indiv_attacker,gtol	string	individual attacker threshold, default is 5% of VDD
glitch_check,latch_limit	string	latch failure criteria,tool's default is 25% of VDD
glitch_check,clock_limit	string	clock failure criteria,tool's default is 5% of VDD

Tempus Foundation Flow User Guide

Tcl Variables

glitch_check,data_limit	string	data failure criteria,tool's default is 80% of VDD
glitch_check,dc_tolerance	string	DC tolerance failure criteria,tool's default is 40% of VDD
Variables for STA Reports		
compress_sta_rpt	string	set to "true" to compress sta report files to .gz, default is "false"
sdf_rpt	string	set to "true" to dump out the delay sdf file
Variables for Plug-In Tcl Scripts		
tempus_pre_design_import	string	<path to plug-in tcl file>,executed before design import (optional)
tempus_replace_design_import	string	<path to plug-in tcl file>,executed to replace default design import (optional)
tempus_post_design_import	string	<path to plug-in tcl file>,executed after design import (optional)
tempus_pre_parasitic_load	string	<path to plug-in tcl file>,executed before parasitic load (optional)
tempus_replace_parasitic_load	string	<path to plug-in tcl file>,executed to replace parasitic load (optional)
tempus_post_parasitic_load	string	<path to plug-in tcl file>,executed after parasitic load (optional)
tempus_pre_sta	string	<path to plug-in tcl file>,executed before first report timing (optional)

Tempus Foundation Flow User Guide

Tcl Variables

tempus_replace_sta	string	<path to plug-in tcl file>,executed to replace first report timing (optional)
tempus_post_sta	string	<path to plug-in tcl file>,executed after first report timing (optional)
tempus_pre_si	string	<path to plug-in tcl file>,executed before SI analysis (optional)
tempus_replace_si	string	<path to plug-in tcl file>,executed to replace SI analysis (optional)
tempus_post_si	string	<path to plug-in tcl file>,executed after SI analysis (optional)
tempus_pre_rpt_gen	string	<path to plug-in tcl file>,executed before reports generation (optional)
tempus_replace_rpt_gen	string	<path to plug-in tcl file>,executed to replace reports generation (optional)
tempus_post_rpt_gen	string	<path to plug-in tcl file>,executed after reports generation (optional)
Variables to Skip Each Default Steps		
tempus_skip_design_import	string	set to "true" to skip design import (optional)
tempus_skip_parasitic_load	string	set to "true" to skip parasitic load (optional)
tempus_skip_sta	string	set to "true" to skip first report timing (optional)
tempus_skip_si	string	set to "true" to skip SI analysis (optional)

Tempus Foundation Flow User Guide

Tcl Variables

tempus_skip_rpt_gen	string	set to "true" to skip report generation (optional)
---------------------	--------	--

Example of setup_default.tcl

Sample script for the non-MMMC flow located at the `EXAMPLES/TEMPUS/dtmf/WORK` directory where the following variables are defined:

```
#####
# report file and saved session dir setting
#####
set vars(flow) "default" ; # supported flow -> default, mmmc and cpf
set vars(script_root) "../..SCRIPTS" ; # foundation flow scripts dir (default is
"../SCRIPTS")
set vars(rpt_dir) "rpt" ; # Report directory (default is rpt)
set vars(dbs_dir) "dbs" ; # Saved session directory (default is dbs)
#set vars(saved_db_name) ; # Saved session name (optional, default is
<design_name>.enc)
#####
# DESIGN INITIALIZATION #
#####
set vars(si_analysis) "true" ; # set "true" to include si analysis (default is
"true")
set vars(aocv_flow) "false" ; # set "true" to switch on the aocv analysis
set vars(pba_analysis) "false" ; # set "true" to switch on path based analysis for
violated paths
set vars(aocv_pba_analysis) "false" ; # set "true" to switch on aocv path based
analysis for violated paths
set vars(physical_data) "false" ; # set "true" to include physical data (default
setting is "false"
# to just load the timing related data)

#####
# Variables define,if design is imported from Encounter DB
#####
set vars(load_edi_db) "false" ; # set "true" for loading design from EDI DB
# design saved from TEMPUS session can be restored in the same way
set vars(edi_db_name) "DBS/signoff.enc" ; # EDI database or a conf file (required if
# load_edi_db is true, default is "DBS/signoff.enc" )
#set vars(edi_oa_name) { lib design view } ; # EDI OA database (required if
load_edi_db is true, and the
# database is in OA format
set vars(spef_file) "../..DATA/rc_max.spef.gz" ; # SPEF files
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#-----
# load basic design data (will automatically be loaded when design is restored from
EDI DB)
#-----
# Load in single library set
set vars(libs,timing) "../..GPDk/LIBS/GPDk045/timing/slow.lib" ; # List of slow
(or fast) .libs
#set vars(libs,si) "" ; # List of slow (or fast) .cdfs (required if si_analysis is
true)
# Load in AOCV library
#set vars(libs,aocv) ; # List of aocv libraries
set vars(netlist) "../..DATA/tdsp_core.v.gz" ; # Input netlist
set vars(lef_files) "../..GPDk/LIBS/GPDk045/gscilib045.lef" ; # Input lef file
(optional, required only if need to load physical data)
#set vars(def_files) ; # Input def file (optional, required only if need to load
physical data)
#set vars(oa_Ref) ; # Input OA_reference_libs (optional, required only if need to
load physical OA data)
set vars(design) "tdsp_core" ; # top design name
set vars(signoff_sdc) "../..DATA/prects.sdc ../..DATA/postcts.sdc ../..DATA/
prop.sdc" ; # signoff SDC file
#####
# distribute enviorement setting
#####
#set vars(distribute) ; # the value can be <local | custom | lsf | rsh>
#set vars(remote_hosts) ; # set to value = Total(setup views + hold views) machines
for distribute all
# analysis views onto different mahcines;
# it can also be set to any value < Total(setup views + hold views)
# the runtime is longer (will be ready before 9lrtm, SI results in this mode is not
accurate now)
#set the below, if distribute is set to rsh
#set vars(rsh,host_list) ; # provide the host name
#set vars(distribute_timeout) ; # provide the time limit for a rsh remote host to
respond, the tool's default is 5 sec
#set the below, if distribute is set to custom
#set vars(custom,script) ; # provide the custom script
#set the below, if distribute is set to lsf
#set vars(lsf,queue) ; # provide the lsf queue name
#set vars(lsf,resource) ; # provide the lsf resource name
#set vars(lsf,args) ; # provide other lsf args
set vars(dsta) "false" ; # by defaultl DMMMC flow will be generated
# set to true to switch on DSTA flow
#####
```

Tempus Foundation Flow User Guide

Tcl Variables

```
# Multithreading Settings
#####
#set vars(local_cpus) ; # specify the number of local CPUs used for multithreading
runs
#####
# Load the other Files if used
#####
#set vars(sdf_file,full) ; # provide the full sdf file name (optional)
#set vars(sdf_file,incr) ; # provided the sdf file name of incr delay (optional)
#set vars(ir_drop) ; # provide the ir-drop files name (optional)
#set vars(read_ilm,cell_list) ; # provide the corresponding ilm cell name list if
ILM herarchical flow is used (optional)
#set vars(read_ilm,dir_list) ; # provide the ilm dir list for the above dir
(optional)
#set vars(write_ilm_dir) ; # provide the writen ilm dir name if want to dump ILM
data (optional)
#####
# ANALYSIS SETTINGS #
#####
#####
# Set operating condition. If not specified,each library will use it's own
condition,with no scaling
#####
# If read single library set
#set vars(op_cond,name) ; # provide the condition name will be used for the
design,e.g "WCCOM" (optional)
#set vars(op_cond,lib) ; # provide the library name where the condition is defined
(optional)
# If restoring from EDI db, which contains min/max library sets
#set vars(op_cond_max,name) ; # provide the max condition name will be used for the
design,e.g "WCCOM" (optional)
#set vars(op_cond_max,lib) ; # provide the max library name where the condition is
defined (optional)
#set vars(op_cond_min,name) ; # provide the min condition name will be used for the
design,e.g "WCCOM" (optional)
#set vars(op_cond_min,lib) ; # provide the min library name where the condition is
defined (optional)
#####
# STA settings
#####
set vars(analysis_mode) "onChipVariation" ; # value can be chosen from single,bcwc
and onChipVariation
# default value for signoff is "onChipVariation"
set vars(cppr) "both" ; # value can be none,setup,hold and both
# default value for signoff is "both"
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#set vars(aocv_analysis_mode) "launch_capture" ; # controls AOCV analysis modes,
other two choices are
# "clock_only" & "separate_data_clock"
#set vars(aocv_derate_mode) "aocv_multiplicative"; # controls the AOCV derating
mode, another choice is "aocv_additive"
#set vars(aocv_derate_refpoint) "1.0" ; # changes to 0 for those AOCV libs reference
point is not 1
set vars(clock_uncertainty,setup) 0.01 ; # set clock uncertainty for setup check
(optional)
set vars(clock_uncertainty,hold) 0.01 ; # set clock uncertainty for hold check
(optional)
#-----
# derate settings (can either be restored from EDI DB, or user setting)
#-----
set vars(derate,cell_data_late) 1.05 ; # set derate value for cell for data late
path (optional)
set vars(derate,cell_data_early) 0.95 ; # set derate value for cell for data early
path (optional)
set vars(derate,cell_clock_late) 1.05 ; # set derate value for cell for clock late
path (optional)
set vars(derate,cell_clock_early) 0.95 ; # set derate value for cell for clock early
path (optional)
set vars(derate,net_data_late) 1.05 ; # set derate value for net for data late path
(optional)
set vars(derate,net_data_early) 0.95 ; # set derate value for net for data early
path (optional)
set vars(derate,net_clock_late) 1.05 ; # set derate value for net for clock late
path (optional)
set vars(derate,net_clock_early) 0.95 ; # set derate value for net for clock early
path (optional)
#set vars(derate_tcl) ; # alternatively, users can specify a derating files
(optional)

#####
# STA reports
#####
set vars(compress_sta_rpt) "false" ; # set to "true" to compress sta report files
to .gz, default is "false"
set vars(sdf_rpt) "true" ; # set to "true" to dump out the delay sdf file
#-----
# SI settings (will automatically be set when design is restored from EDI DB in
91rtm)
#-----
set vars(si_rpt_prefix) "tempus" ; # set the SI report prefix, default is "tempus"
set vars(process) "65nm" ; # set SI analysis process,e.g. "65nm"
#set vars(noise,delta_absolute) ; # set the noise delay threshold,the tool's
default value is 0.0
```


Tempus Foundation Flow User Guide

Tcl Variables

```
set vars(twf,conv_iter) 2 ; # set the timing window convergence,the tool's default
setting is 2

#set vars(twf_file) ; # set the load in twf file name (optional),otherwise infinite
twf is used to start
#-----
# Advanced SI settings (will automatically be set when design is restored from EDI
DB in 91rtm)
#-----
#set vars(noise,delay_peak) ; # noise peak for noise-on-delay filtering,tool's
default is 20% of the VDD
#set vars(noise,glitch_peak) ; # noise peak for glitch filtering,tool's default is
20% of the VDD
set vars(vir_attacker,mode) "current" ; # specify whether to use the current-
matching based method for
# virtual attackers,default setting is "current"
#set vars(vir_attacker,factor) ; # multiplication factor of virtual attacker,tool's
default is 1
set vars(vir_attacker,gto1) 0.02 ; # virtual attacker grounding threshold, default
is 2% of VDD
#set vars(vir_attacker,switch_prob) ; # virtual switch probability,only valid with
current mode VA;
# tool's default is 0.3333
#set vars(indiv_attacker,gto1) ; # individual attacker threshold, default is 5% of
VDD
#set vars(glitch_check,latch_limit) ; # latch failure criteria,tool's default is
25% of VDD
#set vars(glitch_check,clock_limit) ; # clock failure criteria,tool's default is
5% of VDD
#set vars(glitch_check,data_limit) ; # data failure criteria,tool's default is 80%
of VDD
#set vars(glitch_check,dc_tolerance) ; # DC tolerance failure criteria,tool's
default is 40% of VDD
#####
# Plug-in variables, each should point to a tcl script file
#####
set vars(tempus_pre_design_import) "../PLUG/init.tcl" ; # <path to plug-in tcl
file>,executed before design import (optional)
#set vars(tempus_replace_design_import) ; # <path to plug-in tcl file>,executed to
replace default design import (optional)
#set vars(tempus_post_design_import) ; # <path to plug-in tcl file>,executed after
design import (optional)
#set vars(tempus_pre_parasitic_load) ; # <path to plug-in tcl file>,executed before
parasitic load (optional)
#set vars(tempus_replace_parasitic_load) ; # <path to plug-in tcl file>,executed
to replace parasitic load (optional)
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#set vars(tempus_post_parasitic_load) ; # <path to plug-in tcl file>,executed after
parasitic load (optional)
#set vars(tempus_pre_sta) ; # <path to plug-in tcl file>,executed before first
report timing (optional)
#set vars(tempus_replace_sta) ; # <path to plug-in tcl file>,executed to replace
first report timing (optional)
#set vars(tempus_post_sta) ; # <path to plug-in tcl file>,executed after first
report timing (optional)
#set vars(tempus_pre_rpt_gen) ; # <path to plug-in tcl file>,executed before
reports generation (optional)
#set vars(tempus_replace_rpt_gen) ; # <path to plug-in tcl file>,executed to
replace reports generation (optional)
set vars(tempus_post_rpt_gen) "../PLUG/sta_rpt.tcl" ; # <path to plug-in tcl
file>,executed after reports generation (optional)
#####
# variables to skip each default steps
#####
#set vars(tempus_skip_design_import) false; # set to "true" to skip design import
(optional)
#set vars(tempus_skip_parasitic_load) false; # set to "true" to skip parasitic load
(optional)
#set vars(tempus_skip_sta) false; # set to "true" to skip first report timing
(optional)
#set vars(tempus_skip_rpt_gen) false; # set to "true" to skip report generation
(optional)
Example of setup_mmmc.tcl
Sample script for the MMMC flow located at the EXAMPLES/TEMPUS/dtmf_mmmc/WORK
directory where the following variables are defined:
#####
# report file and saved session dir setting
#####
set vars(flow) "mmmc" ; # supported flow -> default, mmmc and cpf
set vars(script_root) "../SCRIPTS" ; # foundation flow scripts dir (default is
"../SCRIPTS")
set vars(dbs_dir) "dbs" ; # Saved session directory (default is dbs)
#####
# DESIGN INITIALIZATION #
#####
set vars(si_analysis) "true" ; # set "true" to include si analysis (default is
"true")
set vars(eco_fix) "false" ; # set "true" to include MS-ECO fixing after timing
analysis is done
# (default is "false"), please note this is only for DMMC flow
set vars(aocv_flow) "false" ; # set "true" to switch on the aocv analysis
set vars(pba_analysis) "false" ; # set "true" to switch on path based analysis for
violated paths
```

Tempus Foundation Flow User Guide

Tcl Variables

```
set vars(aocv_pba_analysis) "false" ; # set "true" to switch on aocv path based
analysis for violated paths
set vars(physical_data) "false" ; # set "true" to include physical data (default
setting is "false"
# to just load the timing related data)
#####
# Variables define,if design is imported from Encounter DB
#####
set vars(load_edi_db) "false" ; # set "true" for loading design from EDI DB
# design saved from TEMPUS session can be restored in the same way
set vars(edi_db_name) "DBS/signoff.enc.dat" ; # EDI database or a conf file
(required if load_edi_db
# is true, default is "DBS/signoff.enc" )
#set vars(edi_oa_name) { lib design view }; # EDI OA database (required if
load_edi_db is true, and the
# database is in OA format
#-----
# load basic design data (will automatically be loaded when design is restored from
EDI DB)
#-----
#set vars(libs,timing) ; # Load all the used .libs when importing the design
(optional from 11.1).
#set vars(libs,si) ; # Load all the used .cdfs when importing the design (optional
from 11.1).
set vars(netlist) "../DATA/tdsp_core.v.gz" ; # Input netlist
set vars(lef_files) "../GPDK/LIBS/GPDK045/gscplib045.lef" ; # Input lef file
(optional, required only if need to load physical data)
#set vars(def_files) ; # Input def file (optional, required only if need to load
physical data)
#set vars(oa_Ref) ; # Input OA_reference_libs (optional, required only if need to
load physical OA data)
set vars(design) "tdsp_core" ; # top design name
#####
# MMMC view definition
#####
set vars(setup_analysis_views) "setup_func_slow " ; # List all analysis views in
this design, e.g. "func_slow test_slow"
# User can use "" here to enable concurrent + distribute MMMC flow, e.g. ""V1 V2"
"V3 V4""
set vars(hold_analysis_views) "hold_func_fast hold_func_typ" ; # List all analysis
views in this design, e.g. "func_fast test_fast"
# User can use "" here to enable concurrent + distribute MMMC flow, e.g. ""V1 V2"
"V3 V4""
#set vars(combine_views) ; # User can also give N views here to enable concurrent
+ distribute MMMC flow
# N is the maximum of views to on the same remote machine
```

Tempus Foundation Flow User Guide

Tcl Variables

```
# provide the spef files foreach rc_corner; use the real rc corner name to replace
$rc_corner here
#set vars($rc_corner,spef_file) "$rc_corner.spef.gz" ; # Load all the used .spef
for the rc_corner, e.g. vars(slow_rc,spef).

# The default spef file name will be $rc_corner.spef.gz
set vars(rc_max,spef_file) "../../DATA/rc_max.spef.gz"

#-----

# detailed view information (will automatically be set when design is restored from
EDI DB)

#-----

set vars(library_sets) "slow_libs typ_libs fast_libs" ; # List all the library sets
used in this design, e.g. "slow fast"

set vars(rc_corners) "rc_max" ; # List all the rc corners used in this design, e.g.
"slow_rc fast_rc"

set vars(delay_corners) "slow_max typ_max fast_max" ; # List all the delay_corner
used in this design, e.g. "slow_dc fast_dc"

set vars(constraint_modes) "func_mode" ; # List all the constraint modes used in
this design, e.g. "func test"

# provide the library files foreach library set; use the real library set name to
replace $library_set here
set vars(slow_libs,timing) "../../GPDk/LIBS/GPDk045/timing/slow.lib" ; # Load all
the used .libs for the library_set, e.g. vars(slow,timing)
set vars(typ_libs,timing) "../../GPDk/LIBS/GPDk045/timing/typical.lib" ;
set vars(fast_libs,timing) "../../GPDk/LIBS/GPDk045/timing/fast.lib" ;

# provide the binding rc_corner and library set name for each delay_corner; use the
real delay corner name to i
# replace $delay_corner here

set vars(slow_max,library_set) slow_libs ; # give the binding library_set name,
e.g. vars(slew_dc,library_set)
set vars(typ_max,library_set) typ_libs ;
set vars(fast_max,library_set) fast_libs ;

set vars(slow_max,rc_corner) rc_max ; # give the binding rc_corner name, e.g.
vars(slew_dc,rc_corner)
set vars(typ_max,rc_corner) rc_max ;
set vars(fast_max,rc_corner) rc_max ;

# provide the binding sdc files for each constraint_mode; use the real constraint
mode name to replace $constraint_mode here
set vars(func_mode,signoff_sdc) "../../DATA/prects.sdc ../../DATA/postcts.sdc ../
../DATA/prop.sdc"; # give the binding sdc_file name, e.g vars(func,sdc_file)

# provide the binding constraint mode and delay corner for each view; use the real
view name to replace $analysis_view here
set vars(setup_func_slow,constraint_mode) func_mode; # give the binding constraint
mode name, e.g vars(func_slow,constraint_mode)
set vars(hold_func_fast,constraint_mode) func_mode ;
set vars(hold_func_typ,constraint_mode) func_mode;
set vars(setup_func_slow,delay_corner) slow_max; # give the binding delay corner
name, e.g. vars(func_slow,delay_corner)
```

Tempus Foundation Flow User Guide

Tcl Variables

```
set vars(hold_func_fast,delay_corner) typ_max;
set vars(hold_func_typ,delay_corner) fast_max;
#set vars(view_tcl) ; # if a predefined view definition file is provided,
# the above MMMC settings are not required
#####
# distribute enviorement setting
#####
#set vars(distribute) rsh ; # the value can be <local | custom | lsf | rsh>
#set vars(remote_hosts) 2 ; # set to value = Total(setup views + hold views)
machines for distribute all
# analysis views onto different mahcines;
# it can also be set to any value < Total(setup views + hold views)
# the runtime is longer (will be ready before 91rtm, SI results in this mode is not
accurate now)
#set the below, if distribute is set to rsh
#set vars(rsh,host_list) "pe-opt8 pe-opt7" ; # provide the host name
#set vars(distribute_timeout) 15 ; # provide the time limit for a rsh remote host
to respond, the tool's default is 5 sec
#set the below, if distribute is set to custom
#set vars(custom,script) ; # provide the custom script
#set the below, if distribute is set to lsf
#set vars(lsf,queue) ; # provide the lsf queue name
#set vars(lsf,resource) ; # provide the lsf resource name
#set vars(lsf,args) ; # provide other lsf args
#####
# Multithreading Settings
#####
#set vars(local_cpus) ; # specify the number of local CPUs used for multithreading
runs
#set vars(remotehost_cpus) ; # if DMMM is enabled, the value means number of CPUs
per remote host
#####
# Load other Files if used for any analysis view
#####
#set vars($analysis_view,sdf_file,full) ; # provide the full sdf file name
(optional)
#set vars($analysis_view,sdf_file,incr) ; # provided the sdf file name of incr
delay (optional)
# in ocv mode both early and late ir_drop need to be specified
#set vars($delay_corner,ir_drop_early) ; # provide the ir-drop files name for early
path (optional)
#set vars($delay_corner,ir_drop_late) ; # provide the ir-drop files name for late
path (optional)
#####
```

Tempus Foundation Flow User Guide

Tcl Variables

```
# ANALYSIS SETTINGS #
#####
#####
# STA settings
#####
set vars(analysis_mode) "onChipVariation" ; # value can be chosen from single,bcwc
and onChipVariation
# default value for signoff is "onChipVariation"
set vars(cpvr) "both" ; # value can be none,setup,hold and both
# default value for signoff is "both"
#set vars($delay_corner,clock_uncertainty,setup) ; # set clock uncertainty for
setup check (optional)
#set vars($delay_corner,clock_uncertainty,hold) ; # set clock uncertainty for setup
check (optional)
#-----
# derate settings (can either be restored from EDI DB, or user setting)
#-----
#set vars($delay_corner,cell_data_late) ; # set derate value for cell for data late
path (optional)
#set vars($delay_corner,cell_data_early) ; # set derate value for cell for data
early path (optional)
#set vars($delay_corner,cell_clock_late) ; # set derate value for cell for clock
late path (optional)
#set vars($delay_corner,cell_clock_early) ; # set derate value for cell for clock
early path (optional)
#set vars($delay_corner,net_data_late) ; # set derate value for net for data late
path (optional)
#set vars($delay_corner,net_data_early) ; # set derate value for net for data early
path (optional)
#set vars($delay_corner,net_clock_late) ; # set derate value for net for clock late
path (optional)
#set vars($delay_corner,net_clock_early) ; # set derate value for net for clock
early path (optional)
#vars($delay_corner,derate_tcl) ; # alternatively, users can specify a derating
files (optional)
#####
# STA reports
#####
set vars(compress_sta_rpt) "false" ; # set to "true" to compress sta report files
to .gz, default is "false"
set vars(sdf_rpt) "true" ; # set to "true" to dump out the delay sdf file
#-----
# SI settings (will automatically be set when design is restored from EDI DB in
91rtm)
#-----
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#set vars(si_rpt_prefix) "tempus" ; # set the SI report prefix,default is "tempus"
set vars(process) "65nm" ; # set SI analysis process,e.g. "65nm"

#set vars(noise,delta_absolute) ; # set the noise delay threshold,the tool's
default value is 0.0

#set vars(twf,conv_iter) 2 ; # set the timing window convergence, tool's default
setting is 2

#set vars($analysis_view,twf_file) ; # set the load in twf file name (optional) for
the specified views,

# otherwise infinite twf is used to start

#-----
# Advanced SI settings (will automatically be set when design is restored from EDI
DB in 91rtm)
#-----

#set vars(noise,delay_peak) ; # noise peak for noise-on-delay filtering,tool's
default is 20% of the VDD

#set vars(noise,glitch_peak) ; # noise peak for glitch filtering,tool's default is
20% of the VDD

#set vars(vir_attacker,mode) "current" ; # specify whether to use the current-
matching based method for

# virtual attackers,default setting is "current"

#set vars(vir_attacker,factor) ; # multiplication factor of virtual attacker,tool's
default is 1

#set vars(vir_attacker,gtol) "0.02" ; # virtual attacker glitch threshold, default
is 2% of VDD

#set vars(vir_attacker,switch_prob) ; # virtual switch probability,only valid with
current mode VA;

# tool's default is 0.3333

#set vars(indiv_attacker,gtol) ; # individual attacker threshold, default is 5% of
VDD

#set vars(glitch_check,latch_limit) ; # latch failure criteria,tool's default is
25% of VDD

#set vars(glitch_check,clock_limit) ; # clock failure criteria,tool's default is
5% of VDD

#set vars(glitch_check,data_limit) ; # data failure criteria,tool's default is 80%
of VDD

#set vars(glitch_check,dc_tolerance) ; # DC tolerance failure criteria,tool;s
default is 40% of VDD

#####
# mmmc eco fix options
#####

set vars(eco_buffers) "" ; # provide buffers for eco fix if $vars(eco_fix) set to
true

# by default, empty cell list is provided to let tool to auto search

set vars(eco_targets) "hold" ; # provide eco optimization targets. Choices include
hold, drv, setup and leakage default is hold

# MS-ECO allows multiple incremental ECOs, e.g. "drv hold setup". Leakage opt needs
separate session.
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#####
# Plug-in variables, each should point to a tcl script file
#####
set vars(tempus_pre_design_import) "../PLUG/init.tcl" ; # <path to plug-in tcl
file>,executed before design import (optional)
#set vars(tempus_replace_design_import) ; # <path to plug-in tcl file>,executed to
replace default design import (optional)
#set vars(tempus_post_design_import) ; # <path to plug-in tcl file>,executed after
design import (optional)
#set vars(tempus_pre_parasitic_load) ; # <path to plug-in tcl file>,executed before
parasitic load (optional)
#set vars(tempus_replace_parasitic_load) ; # <path to plug-in tcl file>,executed
to replace parasitic load (optional)
#set vars(tempus_post_parasitic_load) ; # <path to plug-in tcl file>,executed after
parasitic load (optional)
#set vars(tempus_pre_sta) ; # <path to plug-in tcl file>,executed before first
report timing (optional)
#set vars(tempus_replace_sta) ; # <path to plug-in tcl file>,executed to replace
first report timing (optional)
#set vars(tempus_post_sta) ; # <path to plug-in tcl file>,executed after first
report timing (optional)
#set vars(tempus_pre_rpt_gen) ; # <path to plug-in tcl file>,executed before
reports generation (optional)
#set vars(tempus_replace_rpt_gen) ; # <path to plug-in tcl file>,executed to
replace reports generation (optional)
#set vars(tempus_post_rpt_gen) ; # <path to plug-in tcl file>,executed after
reports generation (optional)
#set vars(tempus_pre_eco) ; # <path to plug-in tcl file>,executed before start eco
opt (optional)
#set vars(tempus_replace_eco) ; # <path to plug-in tcl file>,executed to replace
eco opt (optional)
#set vars(tempus_post_eco) ; # <path to plug-in tcl file>,executed after eco opt
(optional)
#####
# variables to skip each default steps
#####
#set vars(tempus_skip_design_import) false ; # set to "true" to skip design import
(optional)
#set vars(tempus_skip_parasitic_load) false; # set to "true" to skip parasitic load
(optional)
#set vars(tempus_skip_sta) false; # set to "true" to skip first report timing
(optional)
#set vars(tempus_skip_rpt_gen) false; # set to "true" to skip report generation
(optional)
Example of setup_cpf.tcl
Sample script for the CPF flow located at the EXAMPLES/TEMPUS/LowPower/WORK
directory where the following variables are defined:
```


Tempus Foundation Flow User Guide

Tcl Variables

```
#####
# report file and saved session dir setting
#####
set vars(flow) "cpf"
set vars(script_root) "../SCRIPTS"
set vars(dbs_dir) "dbs"

#####
# DESIGN INITIALIZATION #
#####
set vars(si_analysis) "false"
set vars(eco_fix) "false"
set vars(aocv_flow) "false"
set vars(pba_analysis) "false"
set vars(aocv_pba_analysis) "false"
set vars(physical_data) "false"
#####
# Variables define,if design is imported from Encounter DB
#####
set vars(load_edi_db) "false"
set vars(edi_db_name) "./DBS/final_signoff.enc"
#####
# CPF file load in and MMC update
#####
set vars(setup_analysis_views) "AVdefault_WC AVdvfs2_WC"
set vars(hold_analysis_views) "AVdefault_BC AVdvfs2_BC"
set vars(rc_max,spef_file) "../SPEF/rc_max.spef.gz"
set vars(rc_min,spef_file) "../SPEF/rc_min.spef.gz"
#####
# distribute MMC enviorement setting (This section is for Celtic-SI flow)
#####
set vars(distribute) "lsf"
set vars(remote_hosts) 4
set vars(lsf,queue) "lnx64"
set vars(lsf,resource) "rusage \[mem=12000:swp=6000\] select \[OSNAME==Linux &&
OSREL==EE40 && SFIARCH==OPT64 && SPEED>=2590 && SPEED<=2595 && mem> 20000\] -P ICD"
set vars(lsf,args) ""
#####
# STA SETTINGS
#####
set vars(analysis_mode) "onChipVariation"
set vars(cpvr) "both"
```

Tempus Foundation Flow User Guide

Tcl Variables

```
#####
#####
# STA reports
#####
#####
set vars(compress_sta_rpt) "false"
set vars(sdf_rpt) "false"
#####
#####
# SI settings (will automatically be set when design is restored from EDI DB in
91rtm)
#####
#####
set vars(si_rpt_prefix) "tempus"
set vars(process) "65nm"
set vars(noise,delta_absolute) 1e-12
set vars(twf,conv_iter) 3
#####
#####
Advanced SI settings (will automatically be set when design is restored from EDI
DB in 91rtm)
#####
#####
set vars(noise,glitch_peak) 0.18
set vars(vir_attacker,mode) "current"
set vars(vir_attacker,gto1) "0.02"
set vars(vir_attacker,switch_prob) 0.2
set vars(glitch_check,latch_limit) 0.25
set vars(glitch_check,clock_limit) 0.06
set vars(glitch_check,data_limit) 0.7
set vars(glitch_check,dc_tolerance) 0.35
#####
# eco fix options
#####
set vars(eco_buffers) ""
set vars(eco_targets) "hold"
```

Timing Analysis Flow

- Running Timing Analysis Using Foundation Flow
 - Optional Inputs
 - Flow Chart
 - Sample of Generated Code
 - Sample Script for Single View Non-MMMC flow
- Running Concurrent and Distributed Multi-Mode Multi-Corner Analysis Using Foundation Flow
- Running Distributed Static Timing Analysis Using Foundation Flow

Running Timing Analysis Using Foundation Flow

Tempus Foundation Flow supports the timing analysis. To run the timing analysis flow, ensure that the following details are available:

- Design Netlist –Verilog
- Timing Constraint files (SDCs)
- Timing libraries (`.libs`)Parasitic data (SPEF files)
- MMMC view definition or CPF file

Note: You can run the timing analysis flow using an available Innovus database.

Optional Inputs

- SI libraries (`.cdB` for glitch analysis)
- Base/Incr Delay Annotation file(s)
- Instance-based IR-drop Information file(s)
- TWF file(s)

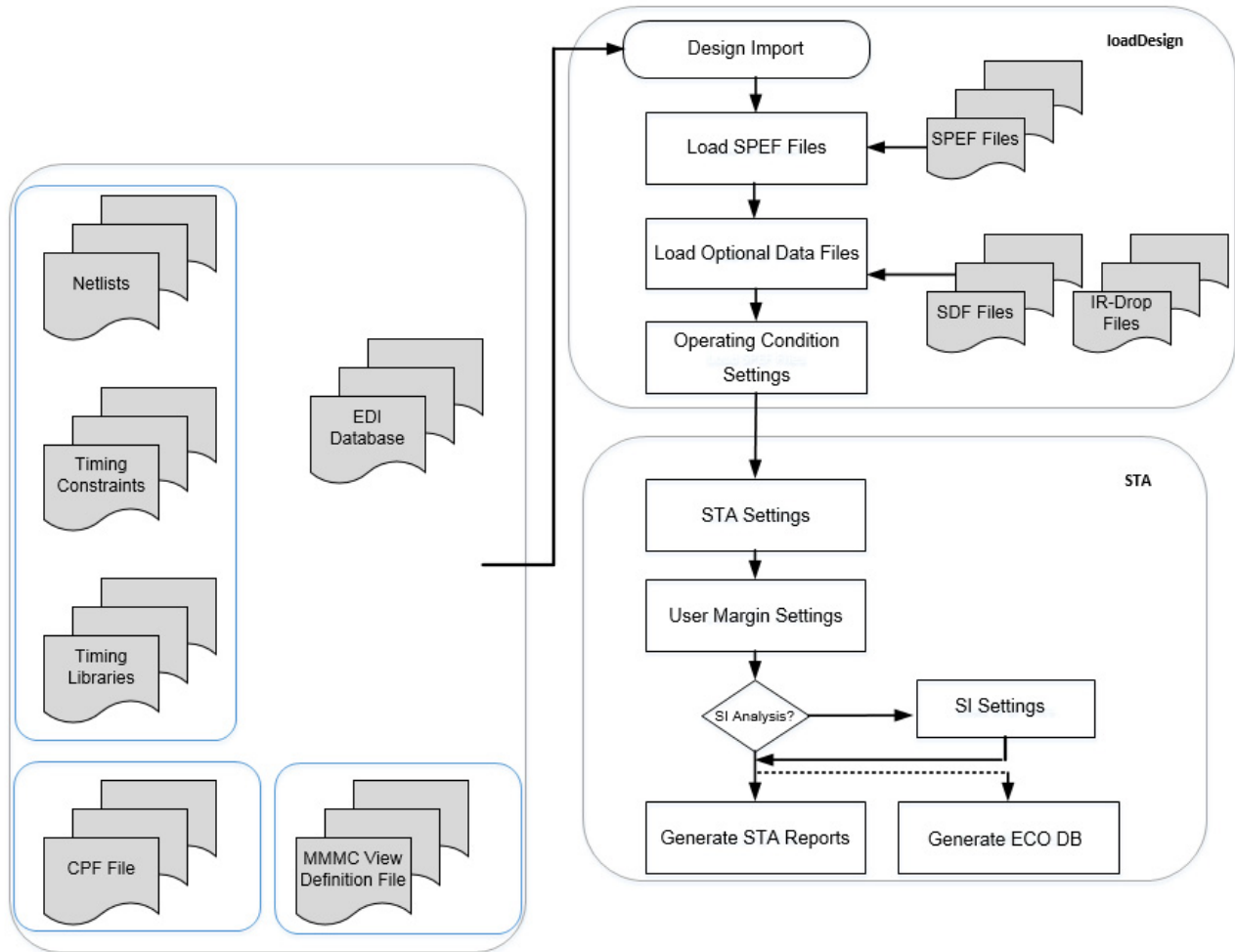
Flow Chart

The figure below illustrates the timing analysis flow.

Tempus Foundation Flow User Guide

Timing Analysis Flow

Figure 3-1 The Timing Analysis Flow



Sample of Generated Code

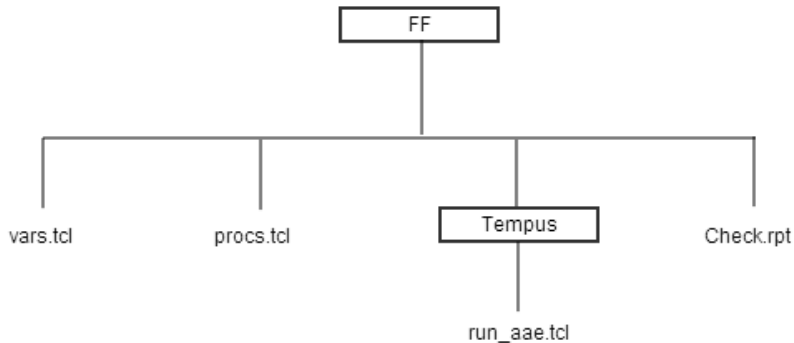
Once you source `gen_tempus_flow.tcl`, the following directory is created under your working directory:

```
FF/ (default)
```

Tempus Foundation Flow User Guide

Timing Analysis Flow

Figure 3-2 The Foundation Flow Directory Structure



Sample Script for Single View Non-MMMC flow

```
# variable and procedure define

source FF/vars.tcl
source FF/procs.tcl
# design import part
read_lib -cdb *.cdb
read_lib *.lib
read_verilog *.v.gz
set_top_module top
read_sdc *.sdc
set aocv_flow false
read_spef *.spef

# STA + SI settings

set_global report_timing_format {instance arc cell load slew delay incr_delay
arrival}
set_global timing_case_analysis_for_icg_propagation true
set_global timing_report_unconstrained_paths true
set_global timing_report_group_based_mode true
set_global timing_cpvr_threshold_ps 10
set_global report_precision 4
set_delay_cal_mode -siAware true -siMode signoff -engine aae
set_si_mode -delta_delay_threshold 20e-12
set_si_mode -individual_attacker_threshold 0.02
set_si_mode -num_si_iteration 2
set_si_mode -accumulated_small_attacker_mode current
report_timing_derate
set_propagated_clock [all_clocks]

# STA report

report_timing
report_analysis_coverage > rpt/report_analysis_coverage.rpt.gz
check_timing -type endpoints -verbose > rpt/uncons_endpoint.rpt.gz
check_timing -type inputs -verbose > rpt/uncons_inputs.rpt.gz
check_timing -type clocks -verbose > rpt/uncons_clocks.rpt.gz
report_constraint -all_violators -late > rpt/constraint_maxdly.rpt.gz
report_constraint -all_violators -early > rpt/constraint_mindly.rpt.gz
report_constraint -all_violators \
```

Tempus Foundation Flow User Guide

Timing Analysis Flow

```
-drv_violation_type max_transition > rpt/constraint_maxtran.rpt.gz
report_constraint -all_violators \
-drv_violation_type max_capacitance > rpt/constraint_maxcap.rpt.gz
report_constraint -all_violators \
-drv_violation_type max_fanout > rpt/constraint_maxfanout.rpt.gz
report_timing -max_paths 500 \
-late \
-path_type full_clock \
-net > rpt/setup.rpt.gz
report_timing -max_paths 500 \
-early \
-path_type full_clock \
-net > rpt/hold.rpt.gz
save_design ./dbs/dtmf_recvr_core.enc -overwrite
write_sdf -interconn noport \
-splitrecrem \
-splitsetuphold rpt/dtmf_recvr_core.3.0.sdf.gz
```

Sample of Predefined Code

Generated code only has valid Tempus commands based on the input settings of `setup.tcl`. Predefined code contains all set of Tempus commands used in different analysis, but with control embedded. For example, the design import part code is shown below:

```
if {[info exists vars(load_edi_db)] && $vars(load_edi_db)} {
    set EDI_DB true
    if $vars(physical_data) {
        if {[info exists vars(edi_oa_name)]} {
            read_design -physical_data -cellview $vars(edi_oa_name)
        } else {
            read_design -physical_data $vars(edi_db_name)
        }
    }
} else {
    ...
}
```

`run_aae.tcl` is the top control script. This script calls `loadDesign.tcl` and `sta.tcl` sequentially.

```
source $vars(script_root)/ETC/TEMPUS/loadDesign.tcl
source $vars(script_root)/ETC/TEMPUS/sta.tcl
```

Running Concurrent and Distributed Multi-Mode Multi-Corner Analysis Using Foundation Flow

Tempus Foundation Flow supports the concurrent and distributed multi-mode multi-corner (MMMC) analysis. For details, see the following sections of the *Tempus User Guide*.

- Concurrent MMMC analysis: See [Concurrent Multi-Mode Multi-Corner Timing Analysis](#)
- Distributed MMMC analysis: See [Distributed Multi-Mode Multi-Corner Timing Analysis](#)

Running Distributed Static Timing Analysis Using Foundation Flow

Tempus Foundation Flow supports the Distributed Static Timing Analysis (DSTA) capability to provide enhanced performance and capacity to real-world designs by using an innovative massive parallel computer architecture. For details on DSTA, see Distributed Static Timing Analysis chapter of the *Tempus User Guide*.

ECO Fixing

- Running ECO Fixing Using Foundation Flow on page 56
 - ❑ Before You Begin on page 56
 - ❑ Flow Chart on page 56
 - ❑ Sample of Generated Code on page 57
 - ❑ Sample Code of Generated run_eco.tcl on page 58
 - ❑ Sample of Predefined Code on page 59

Running ECO Fixing Using Foundation Flow

Tempus Foundation Flow supports the Hold ECO fixing. This is an additional step after the MMMC timing analysis, which Tempus offers to back-end designers for quick final timing closure during the Signoff stage. There are two ways to run hold ECO fixing after the timing analysis:

- Do not exit the session and perform ECO fixing, or
- Save an ECO database and exit the current session; ECO fixing is performed in a new session with previously generated DB.

For better flexibility, it is recommended that you use the second option.

Before You Begin

No additional input files are required to perform ECO fixing after timing analysis.

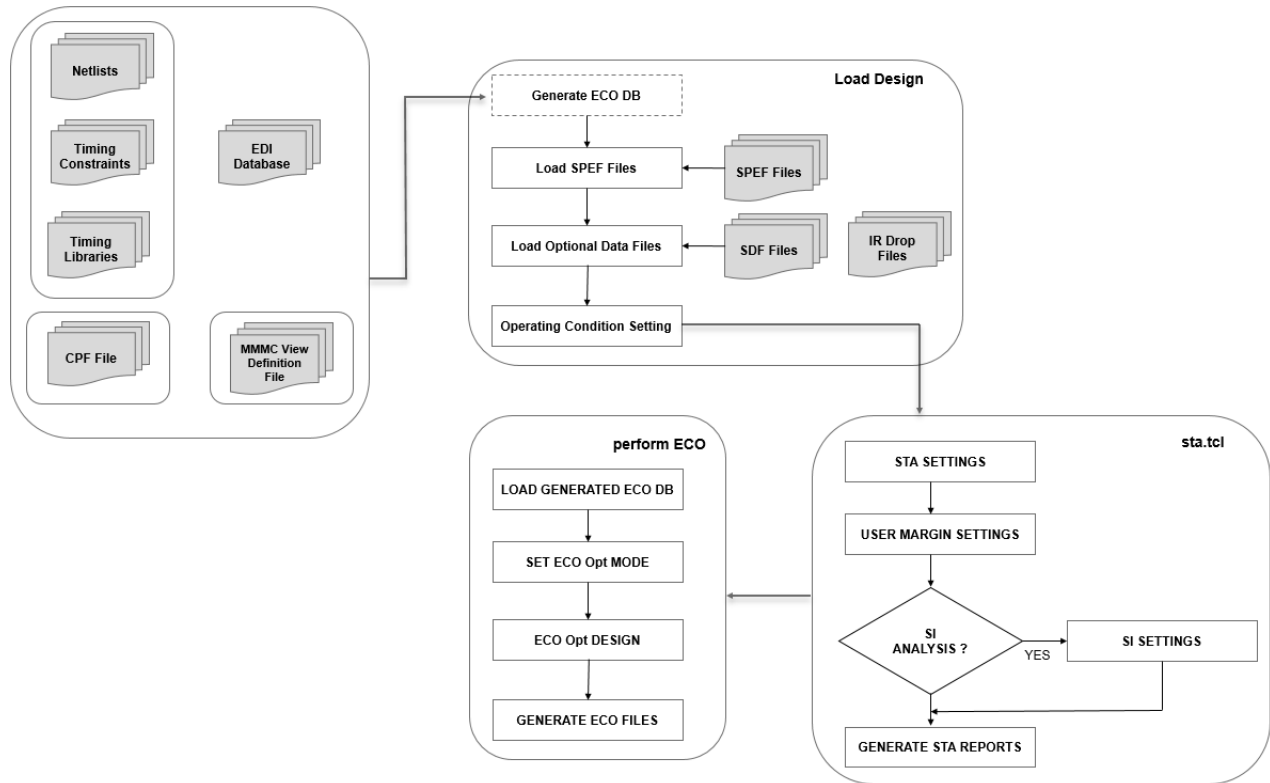
Flow Chart

The figure below illustrates the ECO fixing flow.

Tempus Foundation Flow User Guide

ECO Fixing

Figure 4-1 The ECO Fixing Flow



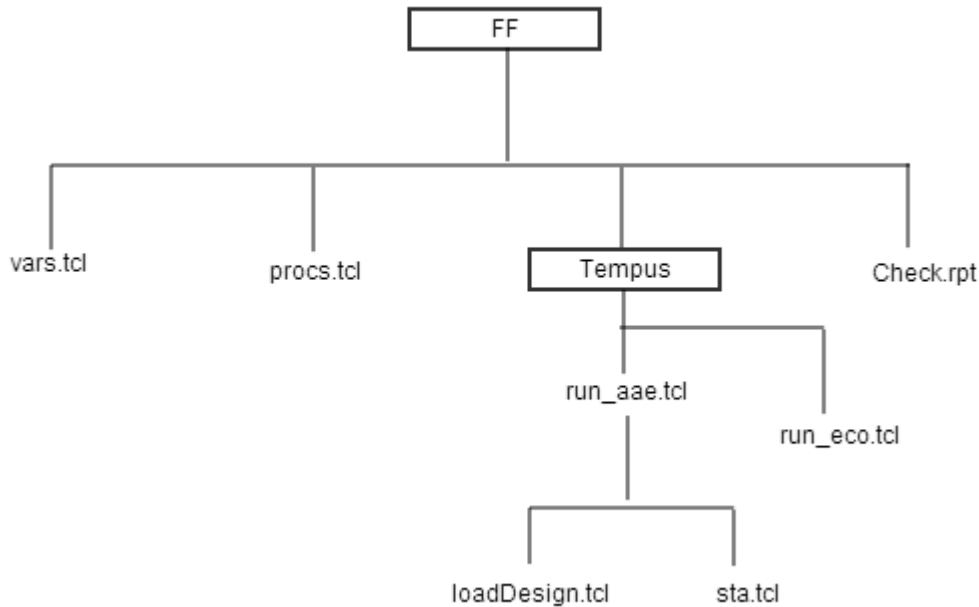
Sample of Generated Code

When you switch on the ECO control variable in the `setup.tcl` file, a `run_eco.tcl` script will be created under the `FF/Tempus` directory, for example:

```
set vars(eco_fix) "true" ;
```

The `run_aae.tcl` file is divided into two scripts, namely `loadDesign.tcl` and `sta.tcl`.

Figure 4-2 FF Directory Structure



Sample Code of Generated run_eco.tcl

To enable eco fixing, set `vars(eco_fix)` to "true". By default, `vars(eco_buffers)` is set to an empty list and ECO target is set to hold in the `setup.tcl` templates. The tool will pick the best buffers from the entire library sets to perform eco fixing. The generated code is presented below:

```
source FF/vars.tcl
source FF/procs.tcl
source FF/TEMPUS/loadDesign.tcl
source FF/TEMPUS/sta.tcl
set_eco_opt_mode -load_eco_opt_db ecoTimingDB
set_eco_opt_mode -verbose true \
    -buffer_cell_list { }
eco_opt_design -hold
```

You can also provide a list of buffers using `vars(eco_buffers)`, for example:

```
set vars(eco_buffers) "BUF_X8 BUF_X1 BUF_X2 BUF_X4 BUF_X16 BUF_X32"
```

The generated `set_eco_opt_mode` command will contain this information, for example:

```
set_eco_opt_mode -verbose true \
```

Tempus Foundation Flow User Guide

ECO Fixing

```
-buffer_cell_list {BUF_X8 BUF_X1 BUF_X2 BUF_X4 BUF_X16 BUF_X32}
```

ECO target can be changed to "setup", "drv", or "leakage" using `vars(eco_targets)`. For example:

```
set vars(eco_targets) "setup"
```

The generated `eco_opt_design` command is shown below:

```
eco_opt_design -setup
```

Signoff ECO allows incremental ECO in the same session. When you set "hold drv setup" to `vars(eco_targets)`, the generated code is shown as follows:

```
set_eco_opt_mode -allow_multiple_incremental true
eco_opt_design -hold
eco_opt_design -drv
eco_opt_design -setup
```

Sample of Predefined Code

The generated code only has valid Tempus commands based on the input settings of `setup.tcl`. Whereas, the predefined code contains all set of Tempus commands used in different analysis, but with control embedded. For example, the sample code of generated `run_eco.tcl` is shown below.

```
source $vars(script_root)/ETC/TEMPUS/loadDesign.tcl
source $vars(script_root)/ETC/TEMPUS/sta.tcl

::FF_TEMPUS::source_plug tempus_pre_eco
if {[info exists vars(tempus_replace_eco)]} {
::FF_TEMPUS::source_plug tempus_replace_eco
} else {
set_eco_opt_mode -verbose true -resize_inst true -optimize_sequential_cells true -
swap_inst true -buffer_cell_list $vars(eco_buffers)
set_eco_opt_mode -load_eco_opt_db ecoTimingDB
if { [llength $vars(eco_targets)] > 1 } { set_eco_opt_mode -
allow_multiple_incremental true }
set i 0
foreach target $vars(eco_targets) {
eco_opt_design -$target
incr i
}
}
::FF_TEMPUS::source_plug tempus_post_eco
```